



UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

Universidad decana de América

Escuela Profesional de Ingeniería de Sistemas

Curso:

Estructura de Datos

2026-1

Semana 12

Árboles B

G. A. Salinas A.

Árboles B

1. Introducción
2. Definición
3. Características
4. Búsqueda
5. Inserción,
6. Eliminación.

Referencias

1. Introducción

ARBOLES MULTICAMINO

En general se denomina ÁRBOLES MULTICAMINO aquellos árboles de grado mayor a dos. Son muy utilizados en la construcción y mantenimiento de *árboles de búsqueda* de gran cantidad de nodos, usualmente almacenados en MEMORIA SECUNDARIA.

El mas elemental de este tipo de árboles son los Árboles B.

Fueron creados por Bayer y McCreight en 1970 y muchos autores sostiene el nombre de arboles B proviene de Bayer.

1. Introduccion

Nueva estructura de datos para gestion de archivos:

Los arboles B son utiles para almacenar y recuperar grandes cantidades de datos, algunas aplicaciones:

- **Arreglos y listas enlazadas.** Estas estructuras para manejo de grandes cantidades de datos resultan ineficientes en medios almacenamiento como discos y cintas.
- **Discos y otros medios similares.** Son medios comunes donde se almacena datos en un sistema de gestion de archivos donde su recuperacion resulta inadecuada por ser lenta.
- **Sistemas de gestion de archivos.** Requieren de una estructura de datos adecuada para trabajar con grandes cantidades de datos en medios de almacenamiento de baja velocidad.
- **Arboles B.** Es la estructura de datos adecuada para gestionar grandes cantidades de datos, demasiado grandes para ser almacenados en medios volatiles (RAM). Estos estan optimizados para operar en medios almacenamiento lento como discos y cintas.

1. Introduccion

Aplicaciones:

Los arboles B son utiles para almacenar y recuperar grandes cantidades de datos, algunas aplicaciones:

- **Bases de datos.** Todas los nodos hoja estan en el mismo nivel, garatiza tiempo de acceso constante independiente del tamano.
- **Sistemas de archivos.** Para organizar y almacenar archivos de manera eficiente.
- **Sistemas operativos.** Para administrar la memoria de manera eficiente.
- **Sistema de Red.** Para enrutar paquetes a traves de la red de manera eficiente.
- **Compiladores.** Para almacenar tablas simbolos para una compilacion eficiente.

1. Introduccion

Aplicaciones:

Los arboles B son utiles para almacenar y recuperar grandes cantidades de datos, algunas aplicaciones:

- Se utiliza en grandes bases de datos para acceder a los datos almacenados en el disco.
- La búsqueda de datos en un conjunto de datos se puede realizar en mucho menos tiempo utilizando un árbol B.
- La mayoría de los servidores también utilizan el enfoque del árbol B.
- Los árboles B se utilizan en sistemas CAD para organizar y buscar datos geométricos.
- Los árboles B también se utilizan en otras áreas como el procesamiento del lenguaje natural, las redes informáticas y la criptografía.

1. Introduccion

Cacteristicas:

Los arboles son utiles para almacenar y recuperar grandes cantidades de datos de forma eficiente.

- **Equilibrado.** Todas los nodos hoja estan al mismo nivel, garatiza tiempo de acceso constante independiente del tamano.
- **Autoequilibrados.** A medida que insertan o eliminan datos el arbol se reestructura de forma automatica
- **Multiples claves por nodo.** Esto permite uso eficienmte de memoria y reduce la altura.
- **Ordenado.** Mantiene el orden de claves en los nodos, hace que busquedas y rangos de consultas sean eficientes.
- **Eficientes.** Son utiles para almacenar y recuperar grandes cantidades de datos

1. Introduccion

Ventajas:

Los arboles son utiles para almacenar y recuperar grandes cantidades de datos de forma eficiente.

- Los árboles B tienen una complejidad temporal garantizada de $O(\log n)$ para operaciones básicas como inserción, eliminación y búsqueda, lo que hace idóneos para grandes conjuntos de datos y aplicaciones en tiempo real.
- Los árboles B son autoequilibrados como los AVL.
- Alta concurrencia y alto rendimiento.
- Utilización eficiente del almacenamiento.

1. Introduccion

Desventajas:

Los arboles son utiles para almacenar y recuperar grandes cantidades de datos pero no todo son ventajas.

- Los árboles B se basan en estructuras de datos almacenadas en disco y pueden consumir mucho espacio y recursos en disco.
- No son la mejor opción para todos los casos, en algunos casos puede ser suficiente el árbol AVL.
- Para conjuntos de datos pequeños, el tiempo de búsqueda en un árbol B puede ser más lento que en un árbol de búsqueda binaria, ya que cada nodo puede contener varias claves.

2. Definición

ÁRBOLES B, árboles de búsqueda, se basan en el criterio de crecimiento de BAYERN-McCREIGHT y se define de la siguiente forma:

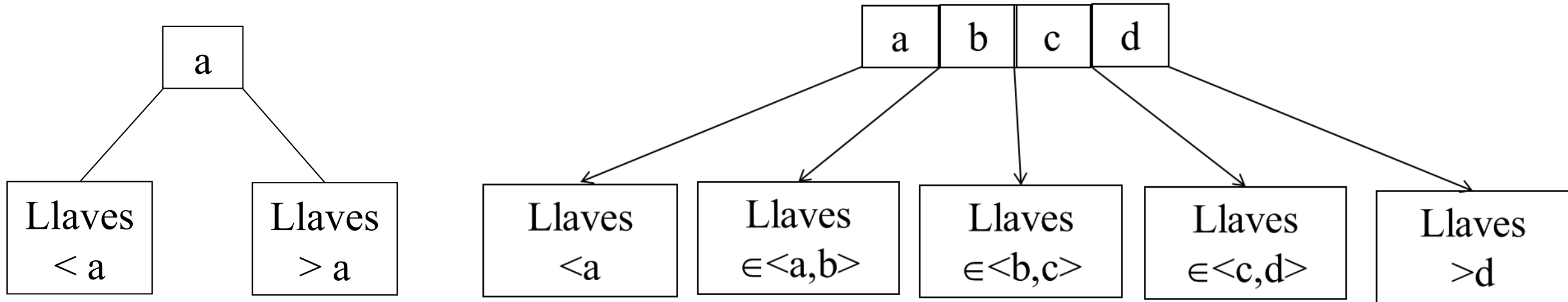
Un árbol B de orden (grado) **n** es aquel árbol de búsqueda que satisface las siguientes propiedades:

- Cada página del árbol B de grado **n** contine como máximo **2n** llaves o claves.
- Cada página contiene como mínimo **n** llaves, excepto la raíz que puede contener una sola llave.
- Cada pagina, es una pagina de hoja o tiene **m+1** descendientes. Donde **m** es el número de llaves en esta pagina.
- Todas las paginas de hoja aparecen al mismo nivel.

2. Definicion

Estructura del Arbol B.

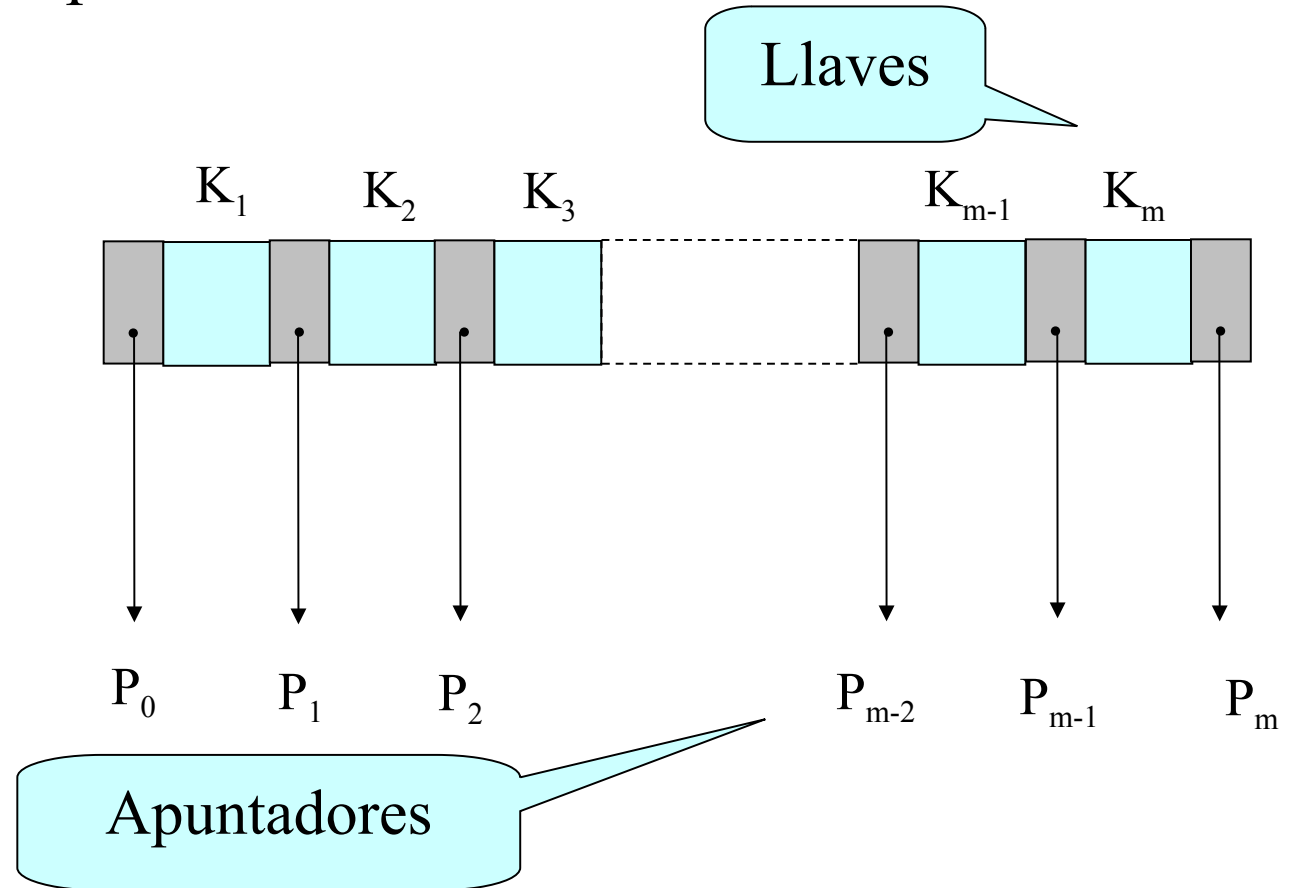
- Los arboles B son una generalización de los árboles balanceados Arbol binario de búsqueda.



2. Definición

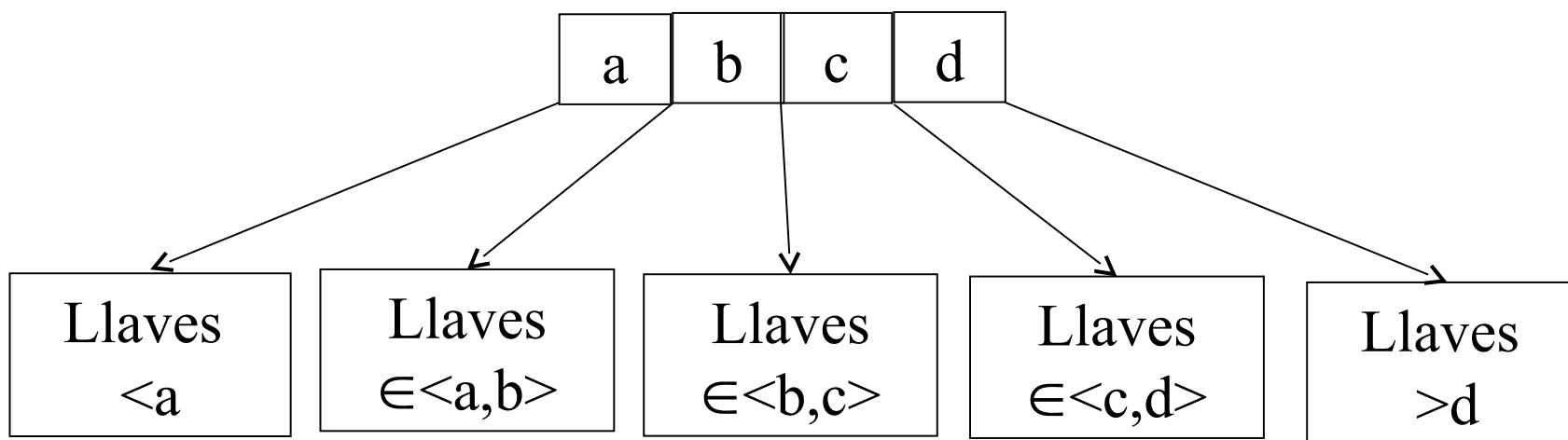
Es lo mas habitual en los árboles de búsqueda. Los árboles B disponen de paginas y se representa con m llaves y $m + 1$ apuntadores.

```
#define N 2
#define MKEY 2*N + 1
struct Pagina {
    TD clave [MKEY]
    Pagina *ramas[MKEY]
};
```

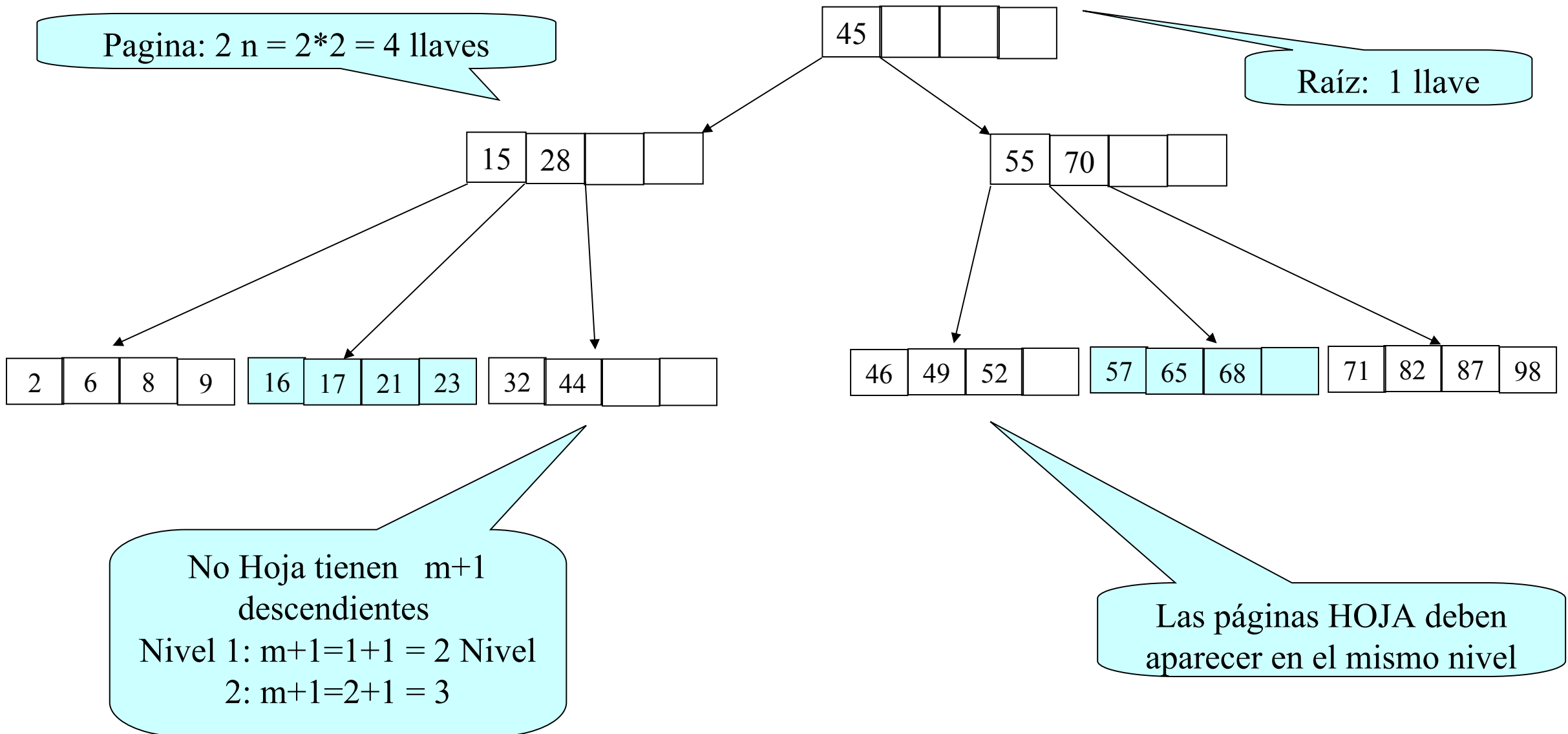


3. Características

- Las llaves o claves dividen el espacio como en un arbol AVL
- Por ejemplo, grado 2: $n=2$: $2n= 2*2 = 4$
 - Numero maximo por pagina: 4 llaves y 5 descendientes.
 - Numero minimo por pagina: 2 llaves y 3 descendientes.
- Se accesa el camino de busqueda similar a un arbol binario balanceado



3. Árbol B de orden 2



4. Búsqueda

- La operacion de busqueda es similar al proceso de busqueda de un arbol ABB. La diferencia es que ahora los nodos son denominados *paginas*.
- En general, las *paginas* de un arbol B se represetan con m llaves y $m+1$ descendientes (apuntadores)

4. Búsqueda

- Seleccionar como nodo actual la raíz del árbol.
- Comprobar si la clave se encuentra en el nodo actual:
 1. Si la clave está, entonces, fin.
 2. Si la clave no está, entonces:
 - Si es en una hoja, no se encuentra la clave. Entonces, Fin.
 - Si no, hacer nodo actual igual al hijo que corresponde según el valor de la clave a buscar y los valores de las claves del nodo actual (buscar la clave K en un nodo con n claves: el hijo izquierdo si $K < K_1$, el hijo derecho si $K > K_n$ y el hijo i -ésimo si $K_i < K < K_{i+1}$) y volver al segundo paso.

5. Insercion

El proceso de insercion requiere tratamiento especial de acuerdo a las características de estos arboles. (Cairo 2006)

- Todas las hojas estan en el mismo nivel y requiere un camino desde la raiz ha alguna de las hojas
- Los arboles B crcen de abajo hacia arriba
- Pasos:

5. Inserción

1. Localizar la pagina que le corresponde de acuerdo al valor de la llave
2. Si el nodo hoja esta lleno (numero de claves igual $m-1$), se procede a la división o re estructuración:
 - Se crea un nuevo nodo, repartiéndose el contenido del nodo lleno entre los dos nodos y la clave intermedia sube de nivel, es decir, se le anade una nueva entrada al padre.
 - Si no esta lleno el padre, se anade la llave, sino se procederá a la división.

5. Re estructuracion

- Dividir la pagina en dos paginas.
- Si la llave $\geq C_{m/2}$, entonces subir un nivel el menor de los mayores de la subpagina y luego hacer:
 - $C_{m/2}$, izquierdo apunta a la subpagina izquierda.
 - $C_{m/2}$, derecho apunta a la subpagina derecha.
- Si la clave $< C_{m/2}$, entonces son simetricas.
- Observacion:
 - La llave $C(m/2)$ puede subir varios niveles.
 - Siguiendo la ruta nodo Hoja-Raiz

5. Inserción

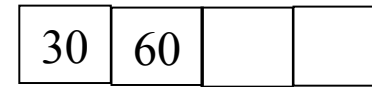
Se tiene un árbol B vacío, insertar los siguientes elementos:

30, 60, 45, 8, 22, 35, 4, 28, 52, 33

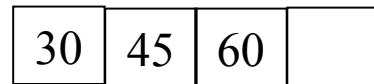
A.



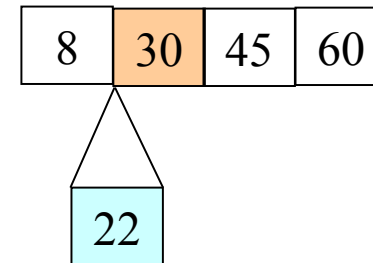
B.



C.



D.

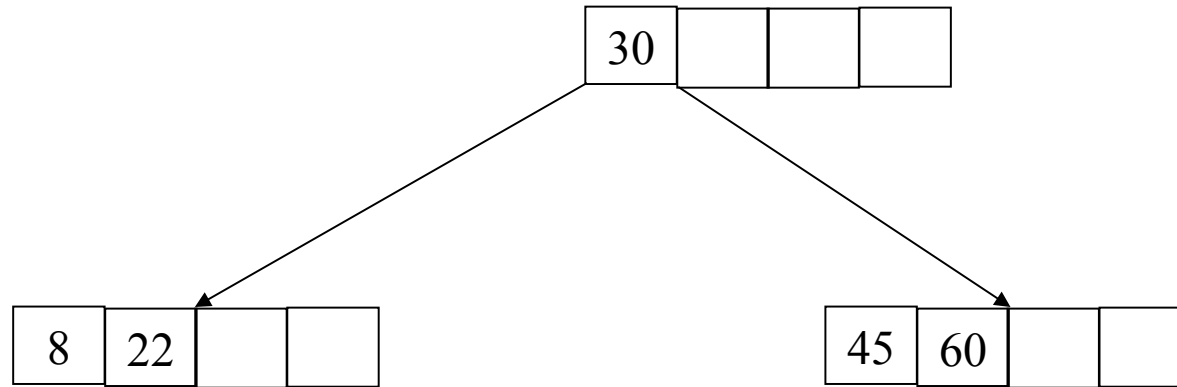


5. Inserción

Se tiene un árbol B vacío, insertar los siguientes elementos:

30, 60, 45, 8, 22, 35, 4, 28, 52, 33

E.

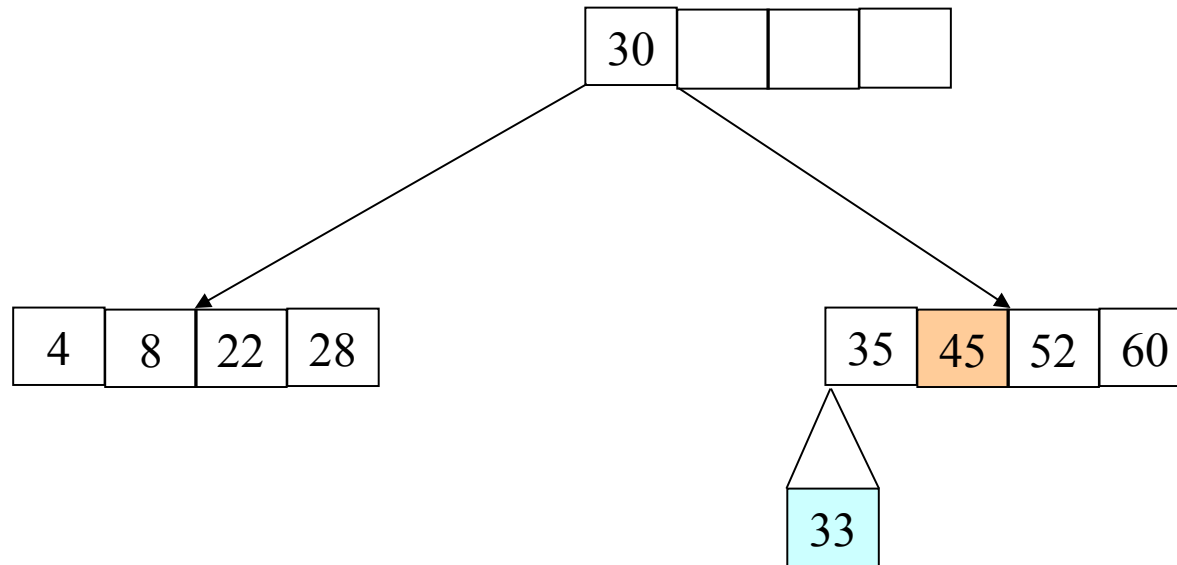


5. Inserción

Se tiene un árbol B vacío, insertar los siguientes elementos:

30, 60, 45, 8, 22, 35, 4, 28, 52, 33

F.

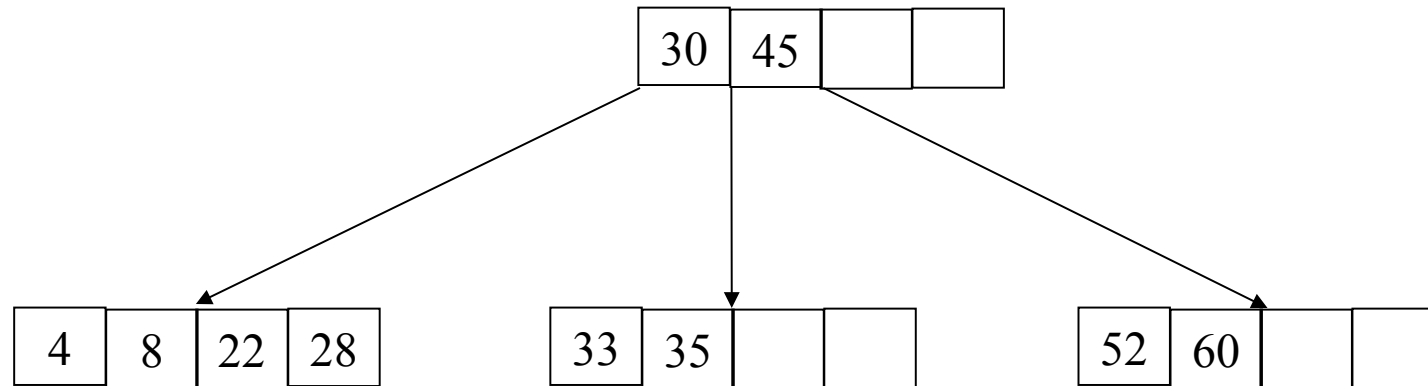


5. Inserción

Se tiene un árbol B vacío, insertar los siguientes elementos:

30, 60, 45, 8, 22, 35, 4, 28, 52, 33, 13, 39, 41, 43, 24, 25, 15

G.



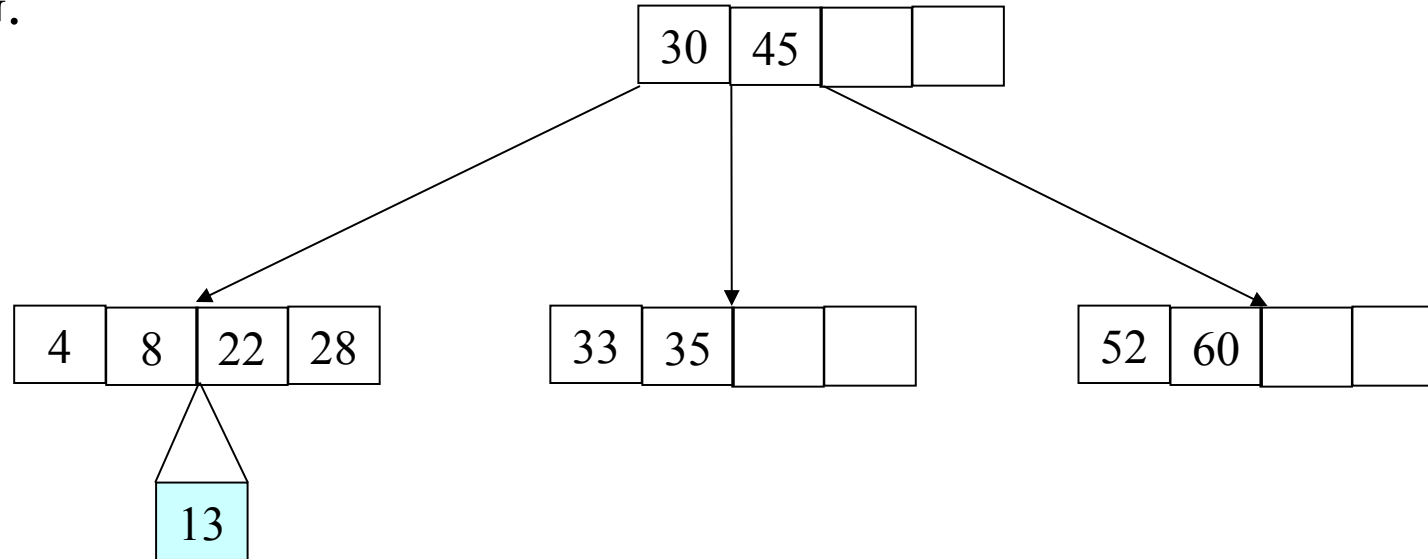
Continúe insertando ...

5. Inserción

Se tiene un árbol B vacío, insertar los siguientes elementos:

30, 60, 45, 8, 22, 35, 4, 28, 52, 33, 13, 39, 41, 43, 24, 25, 15

G.



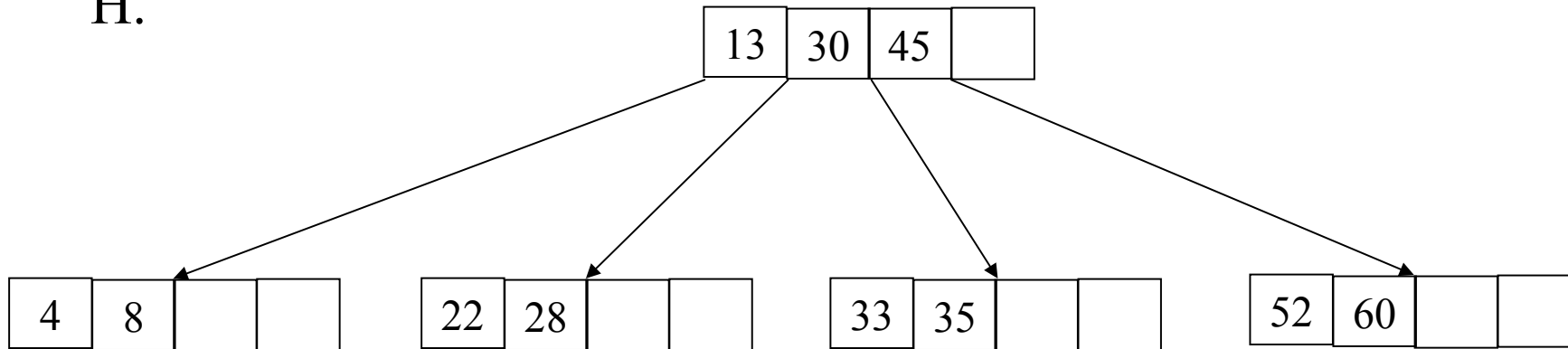
Continúe insertando ...

5. Inserción

Se tiene un árbol B vacío, insertar los siguientes elementos:

30, 60, 45, 8, 22, 35, 4, 28, 52, 33, 13, 39, 41, 43, 24, 25, 15

H.



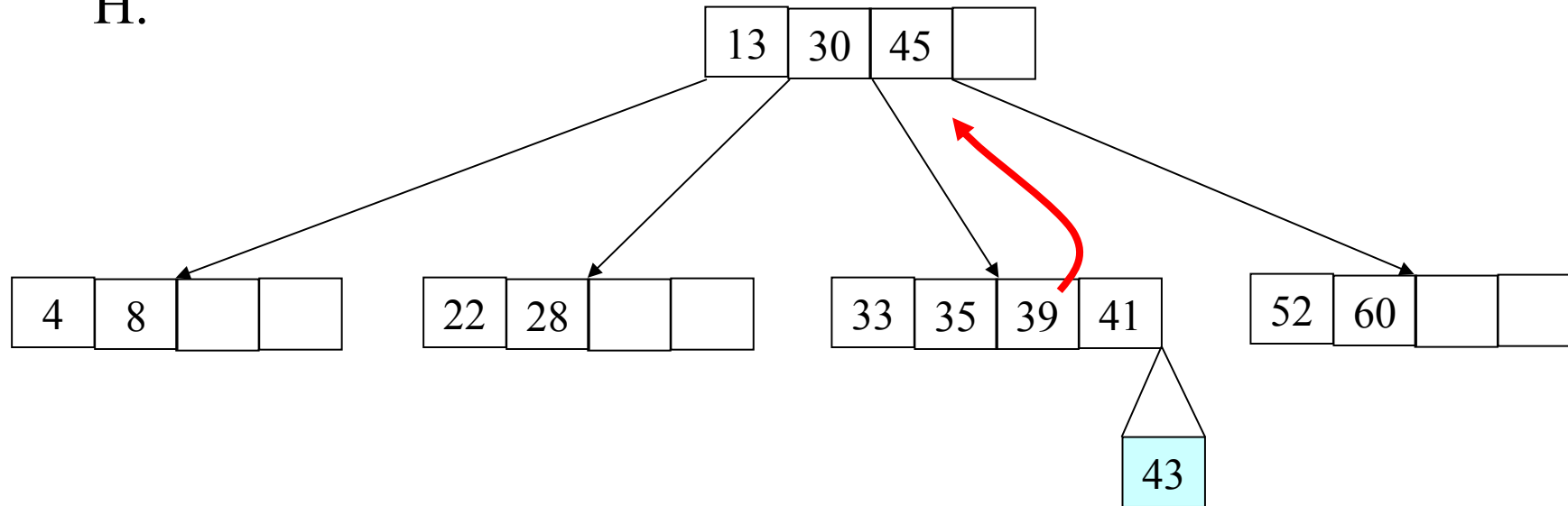
Continúe insertando ...

5. Inserción

Se tiene un árbol B vacío, insertar los siguientes elementos:

30, 60, 45, 8, 22, 35, 4, 28, 52, 33, 13, 39, 41, 43, 24, 25, 15

H.



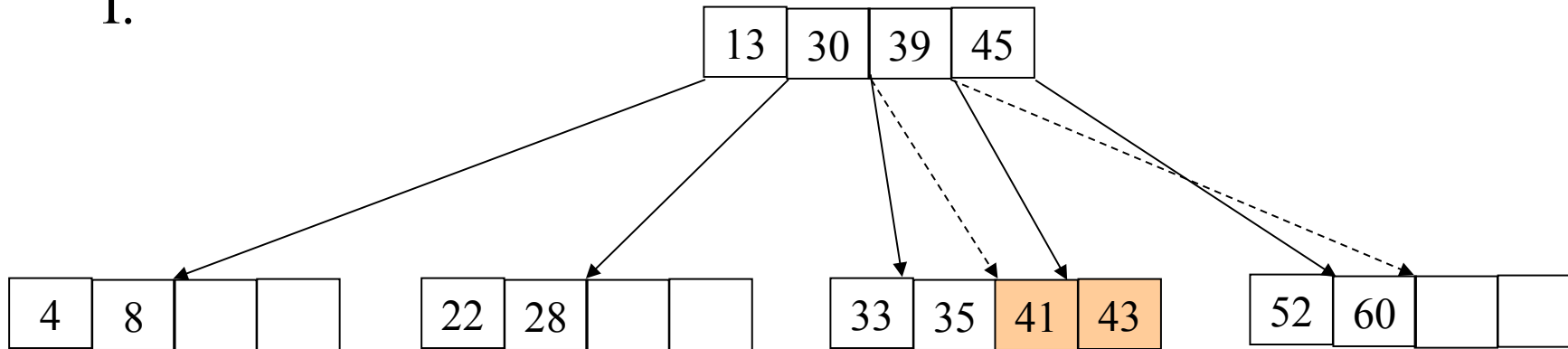
Continúe insertando ...

5. Inserción

Se tiene un árbol B vacío, insertar los siguientes elementos:

30, 60, 45, 8, 22, 35, 4, 28, 52, 33, 13, 39, 41, 43, 24, 25, 15

I.



Cuatro llaves, debe tener
 $m+1$ hojas ($4 + 1$)

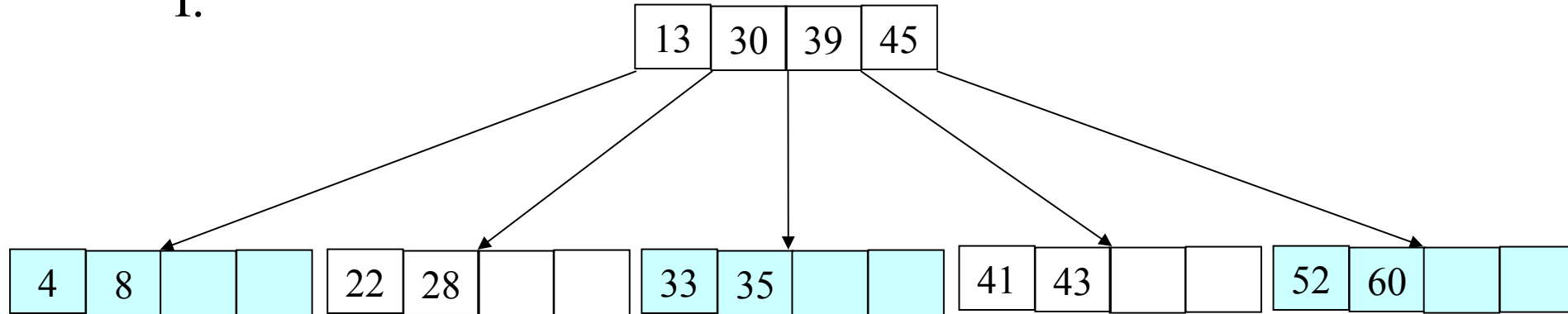
Continúe insertando ...

5. Inserción

Se tiene un árbol B vacío, insertar los siguientes elementos:

30, 60, 45, 8, 22, 35, 4, 28, 52, 33, 13, 39, 41, 43, 24, 25, 15

I.



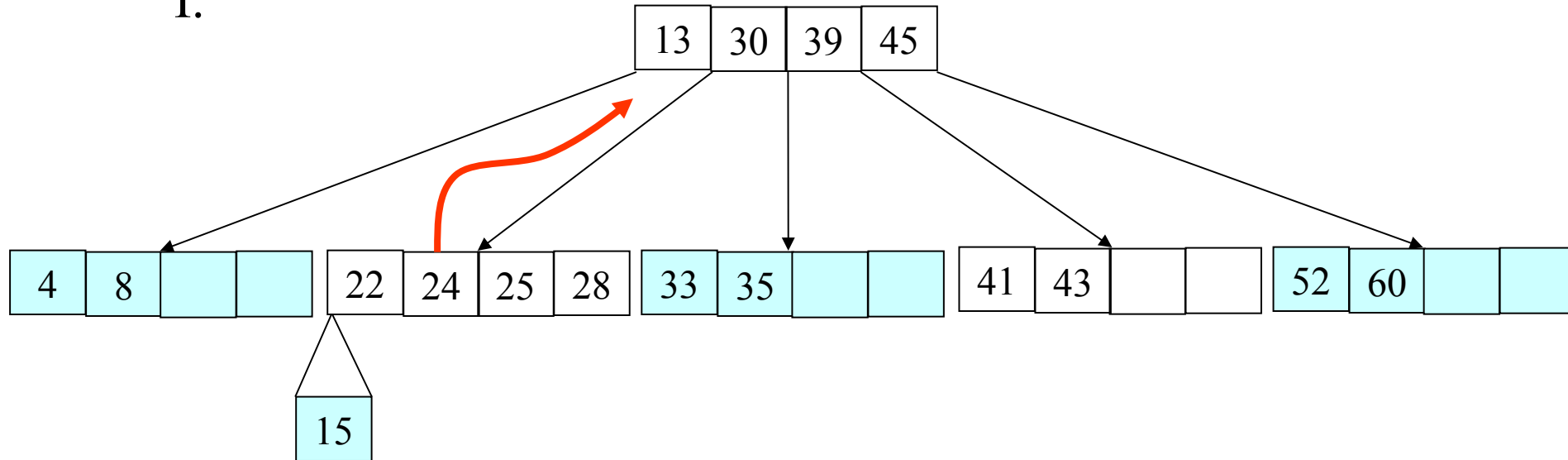
Continúe insertando ...

5. Inserción

Se tiene un árbol B vacío, insertar los siguientes elementos:

30, 60, 45, 8, 22, 35, 4, 28, 52, 33, 13, 39, 41, 43, 24, 25, 15

I.

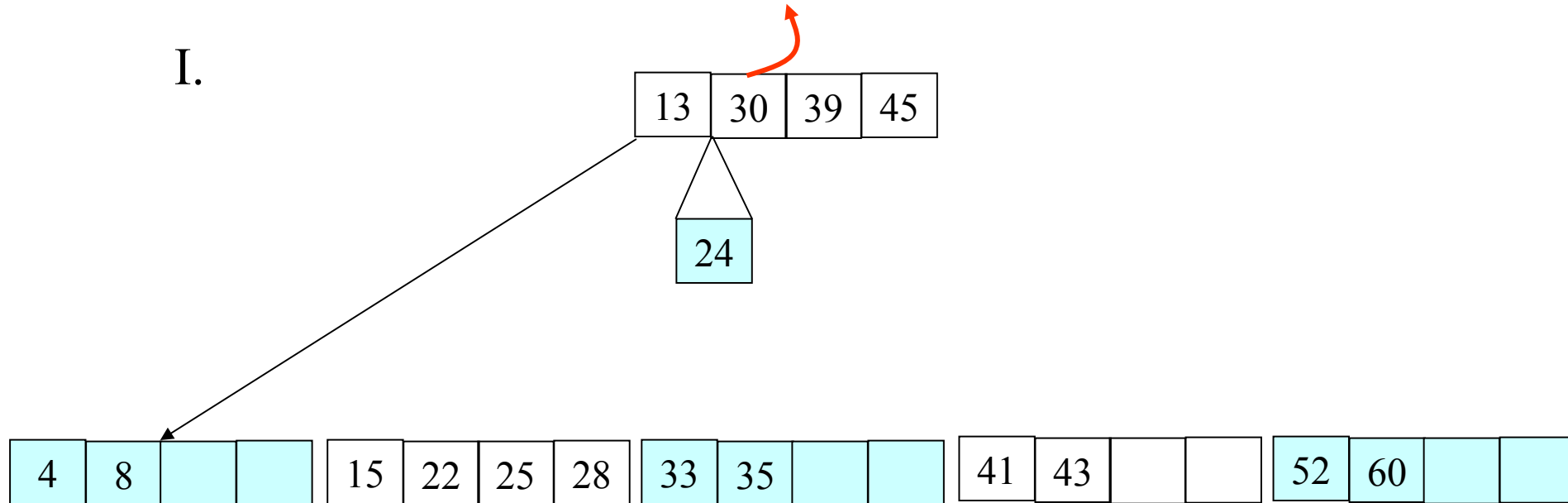


Continúe insertando ...

5. Inserción

Se tiene un árbol B vacío, insertar los siguientes elementos:

30, 60, 45, 8, 22, 35, 4, 28, 52, 33, 13, 39, 41, 43, 24, 25, 15



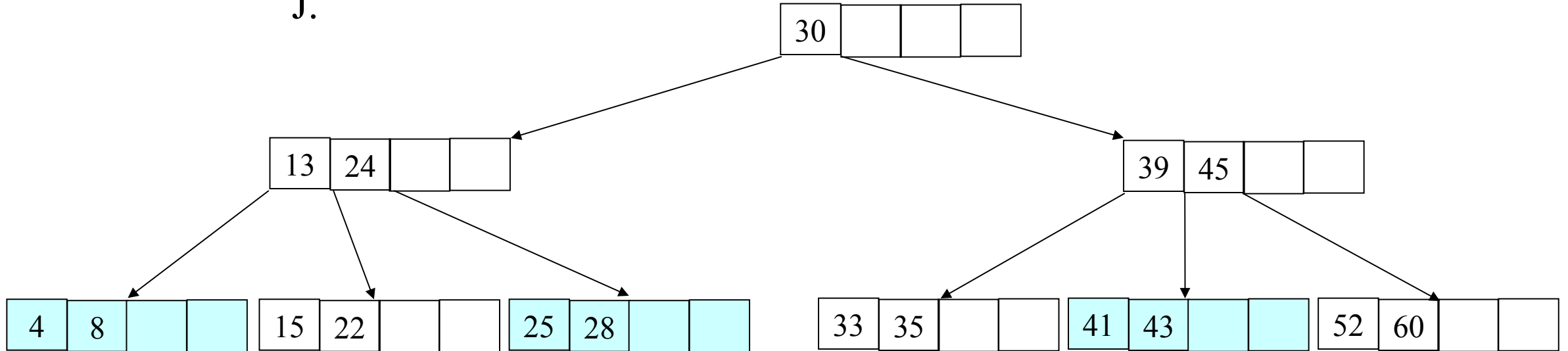
Continúe insertando ...

5. Inserción

Se tiene un árbol B vacío, insertar los siguientes elementos:

30, 60, 45, 8, 22, 35, 4, 28, 52, 33, 13, 39, 41, 43, 24, 25, 15

J.



6. Eliminacion

- Si la llave se encuentra en una pagina hoja, entonces la eliminacion es directa.
- En caso contrario:
 - Intercambiar el elemento con un elemento de una pagina hoja.
 - **Subarbol izquierdo.** Tomar el elemento de mas a la derecha de la pagina hoja del sub arbol izquierdo: ***El mayor de los menores.***
 - **Subarbol derecha.** Tomar el elemento de mas a la izquierda de la pagina hoja del subarbol derecho: ***El menor de los mayores.***

6. Eliminacion

- Si despues del intercambio de eliminacion se produce que una pagina tenga menos de n elementos. Entonces se produce una ***subocupacion***.
- La nueva pagina sera:
 - La pagina actual.
 - La pagina adyacente
 - El elemento entre ellas del nivelsuperior.
- Este proceso se realiza en direccion al nodoRaiz..

6. Eliminación

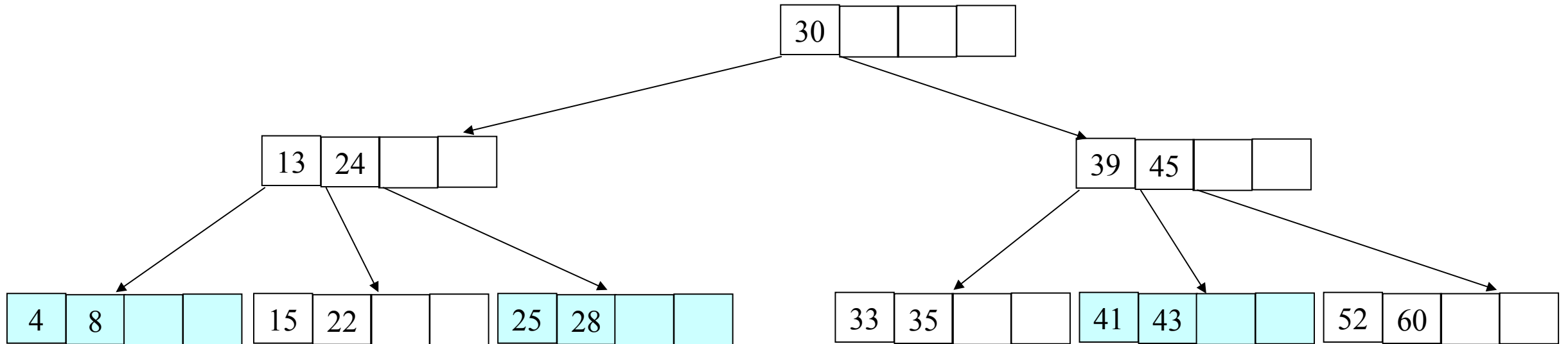
Las operaciones a realizar son:

- **Redistribución.** Se utiliza en el caso en que al borrar una clave el nodo se queda con un número menor que el mínimo y uno de los hermanos adyacentes tiene al menos uno más que ese mínimo.
- **Unión.** Se utiliza en el caso de que no sea posible la redistribución y por tanto sólo será posible unir los nodos junto con la clave que los separa que se encuentra en el padre.

6. Eliminación

Se tiene un árbol B con los siguientes elementos:

Eliminar: 30

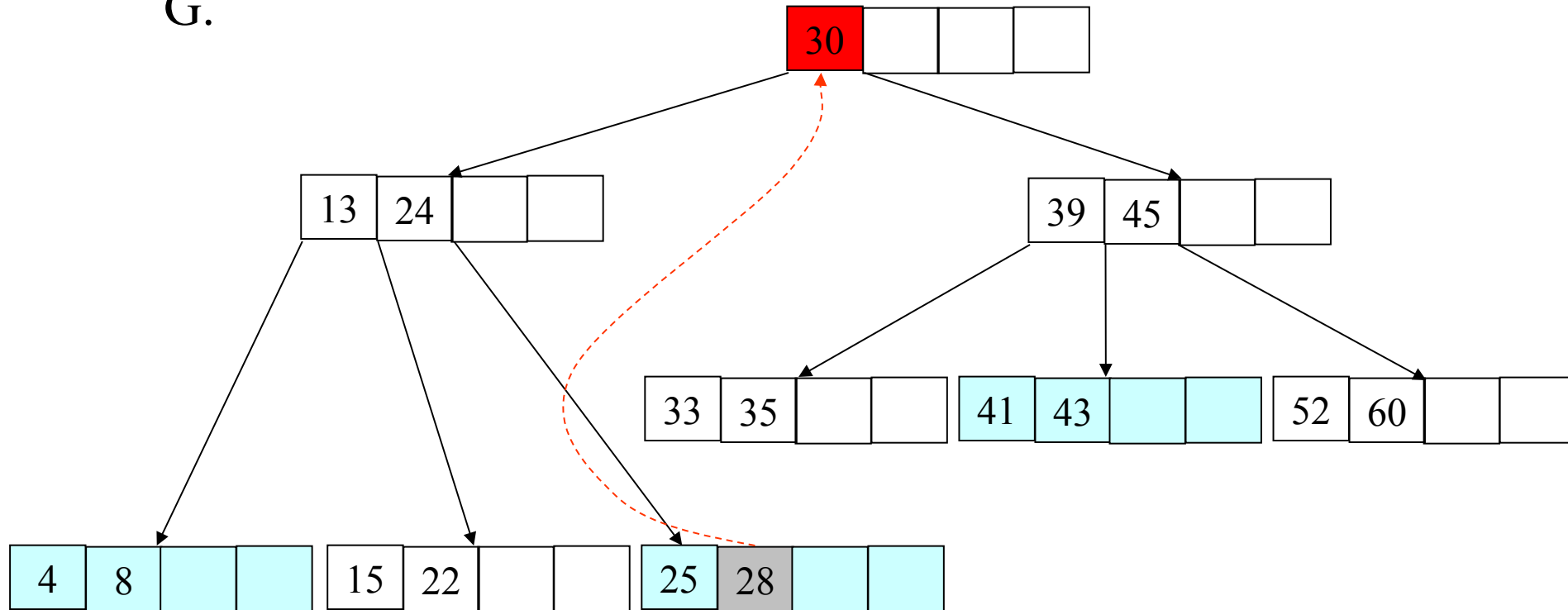


6. Eliminación

Se tiene un árbol B con los siguientes elementos:

Eliminar: 30

G.

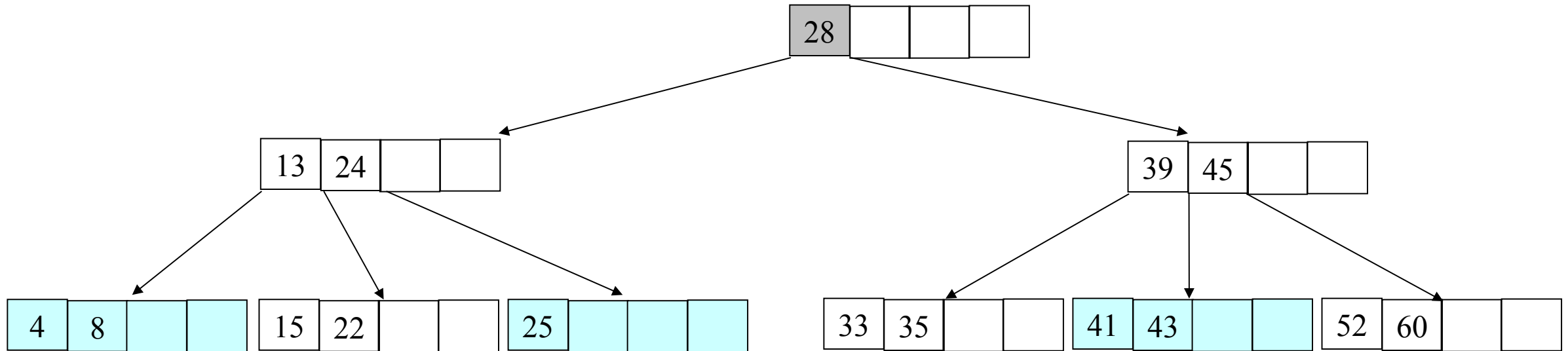


6. Eliminación

Se tiene un árbol B con los siguientes elementos:

Eliminar: 30

Se sustituye el mayor de los menores



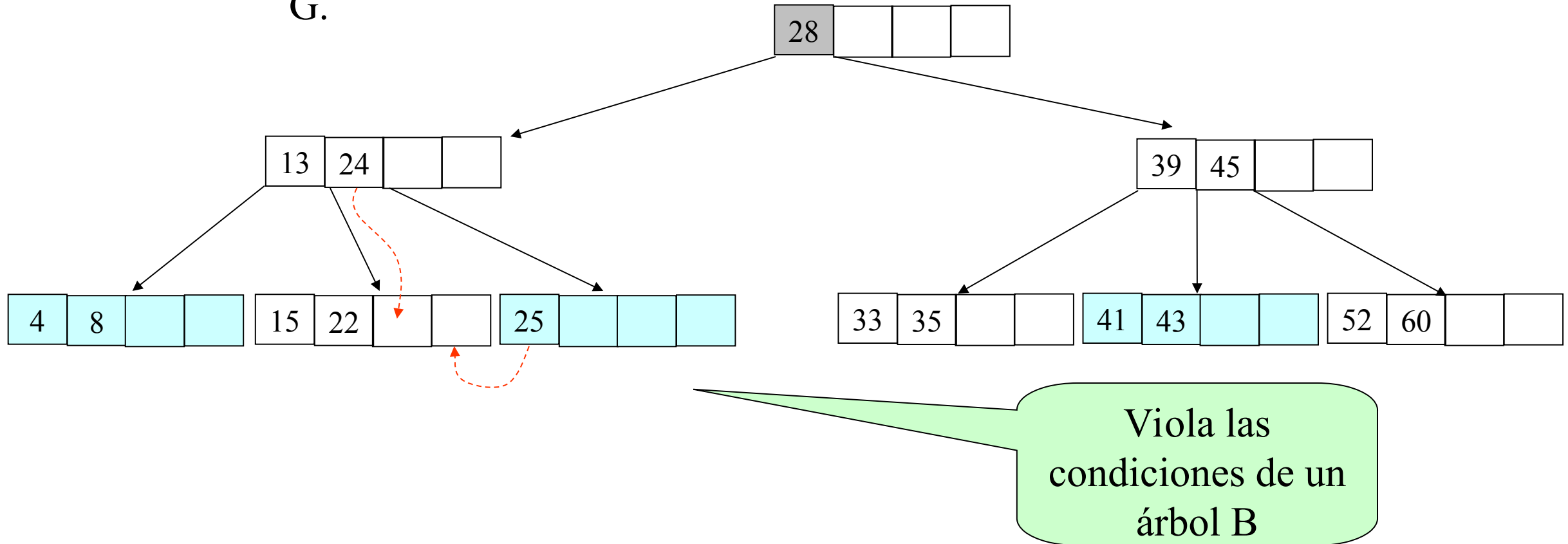
Viola las condiciones de un árbol B

6. Eliminación

Se tiene un árbol B con los siguientes elementos:

Eliminar: 30

G.

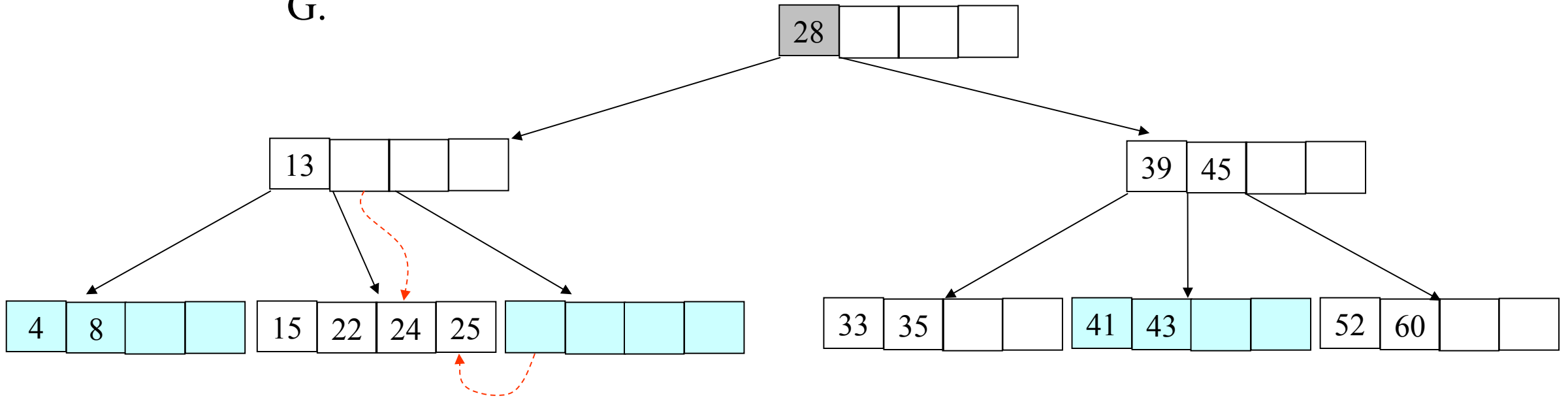


6. Eliminación

Se tiene un árbol B con los siguientes elementos:

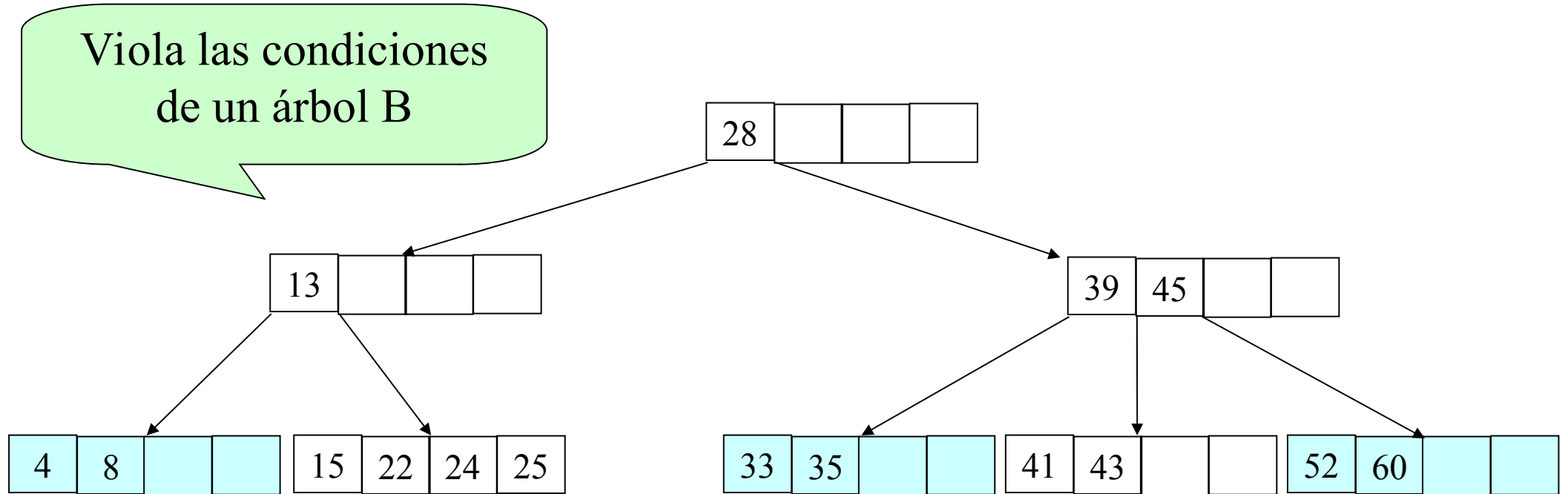
Eliminar: 30

G.



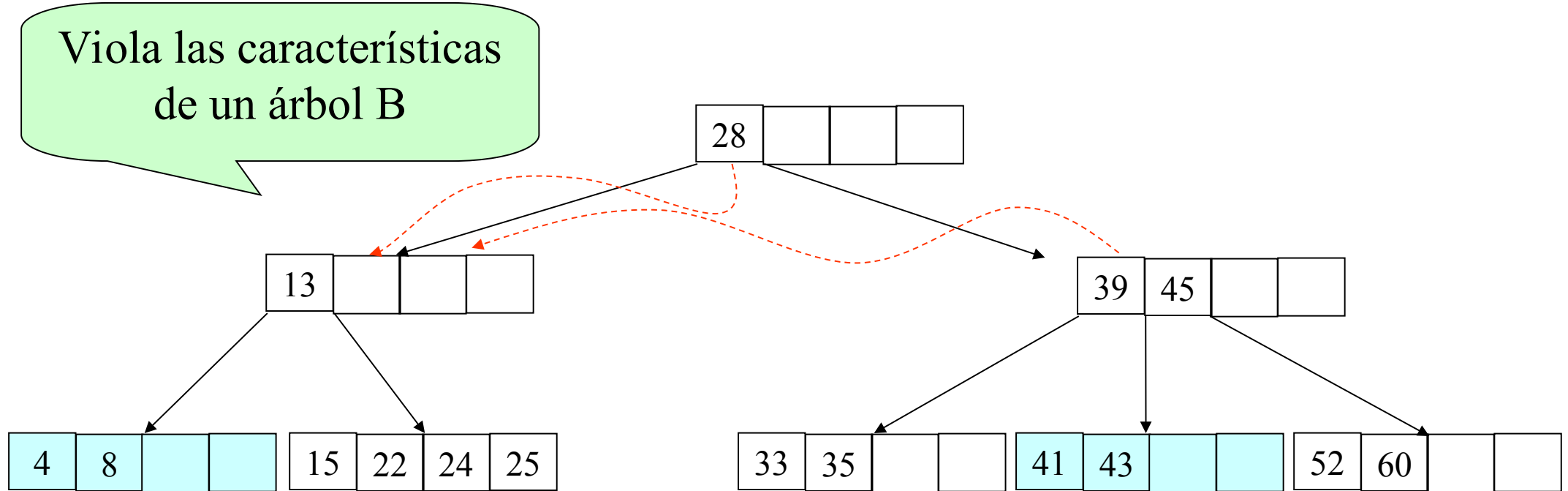
6. Eliminación

Se tiene un árbol B con los siguientes elementos:



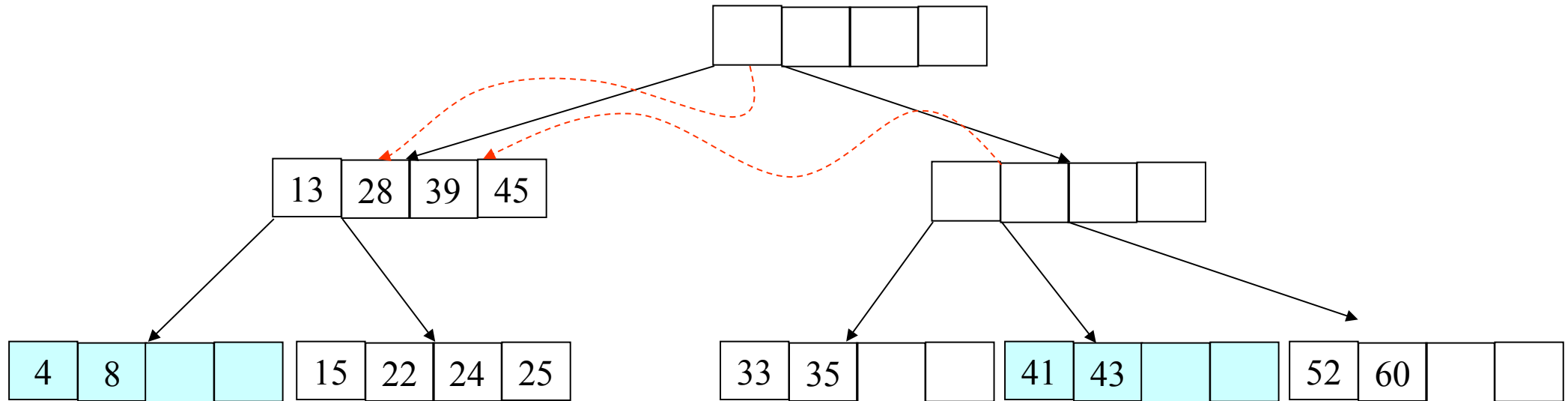
6. Eliminación

Se tiene un árbol B con los siguientes elementos:



6. Eliminación

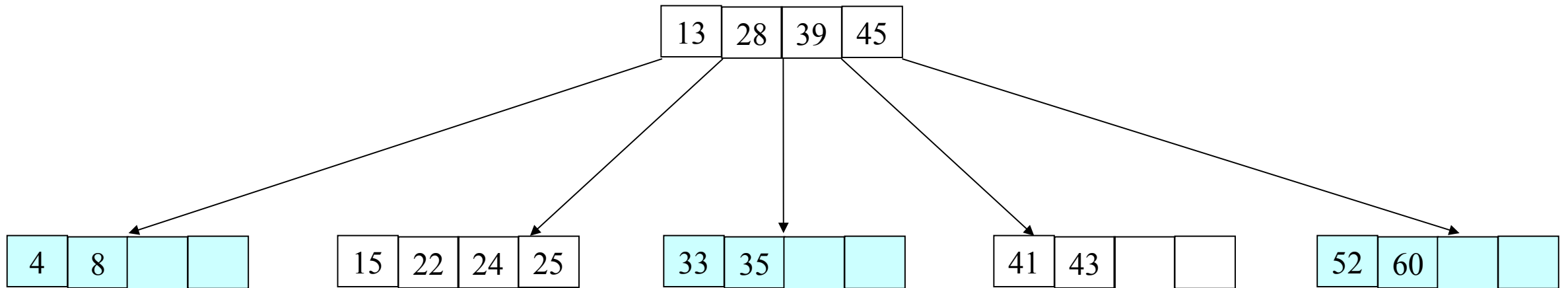
Se tiene un árbol B con los siguientes elementos:



6. Eliminación

Se tiene un árbol B con los siguientes elementos:

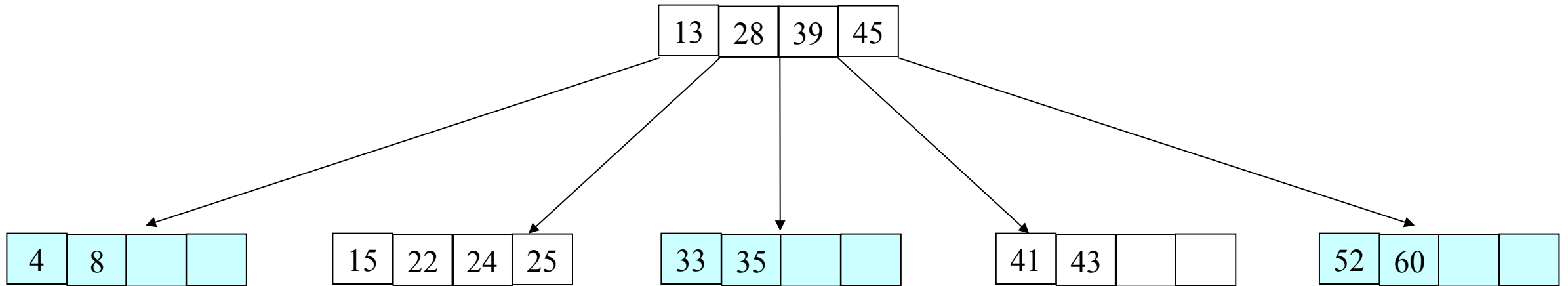
Cumple condiciones de
un árbol B



6. Eliminación

Se tiene un árbol B con los siguientes elementos:

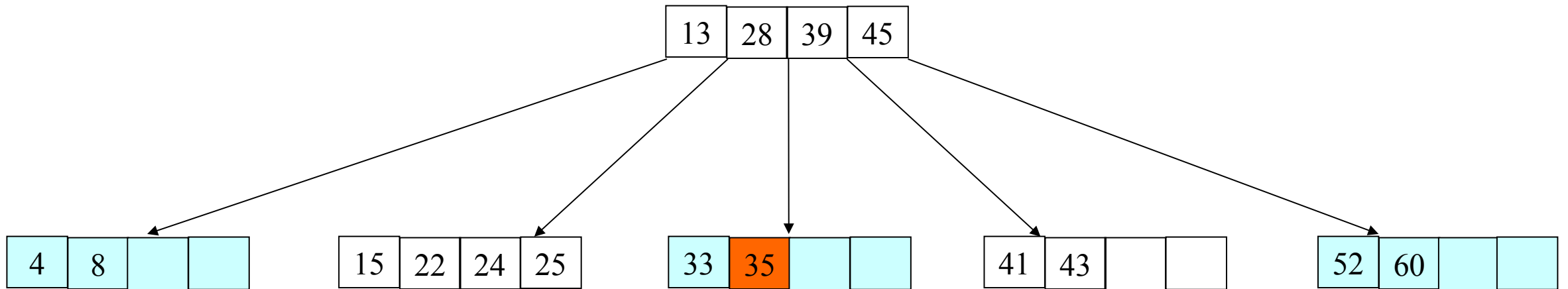
Eliminar: 35



6. Eliminación

Se tiene un árbol B con los siguientes elementos:

Eliminar: 35

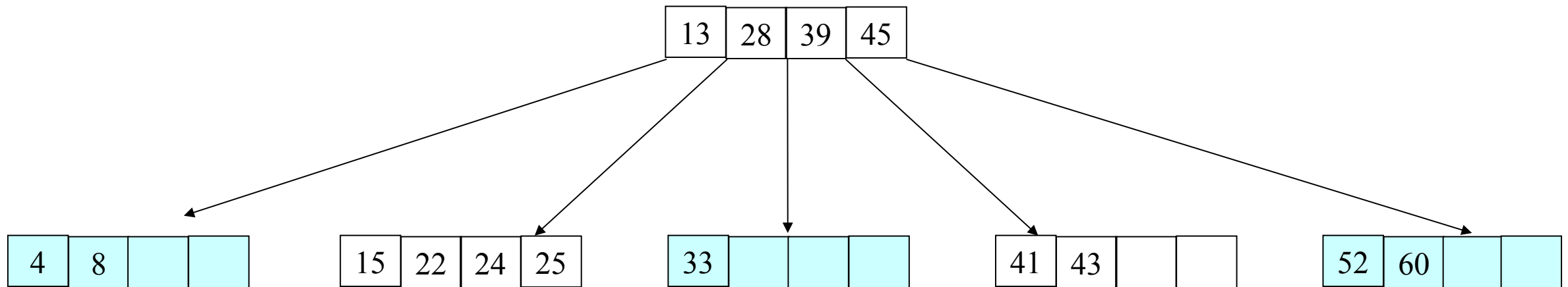


El elemento esta en una
pagina hoja. No es
necesario sustituirlo

6. Eliminación

Se tiene un árbol B con los siguientes elementos:

Eliminar: 35



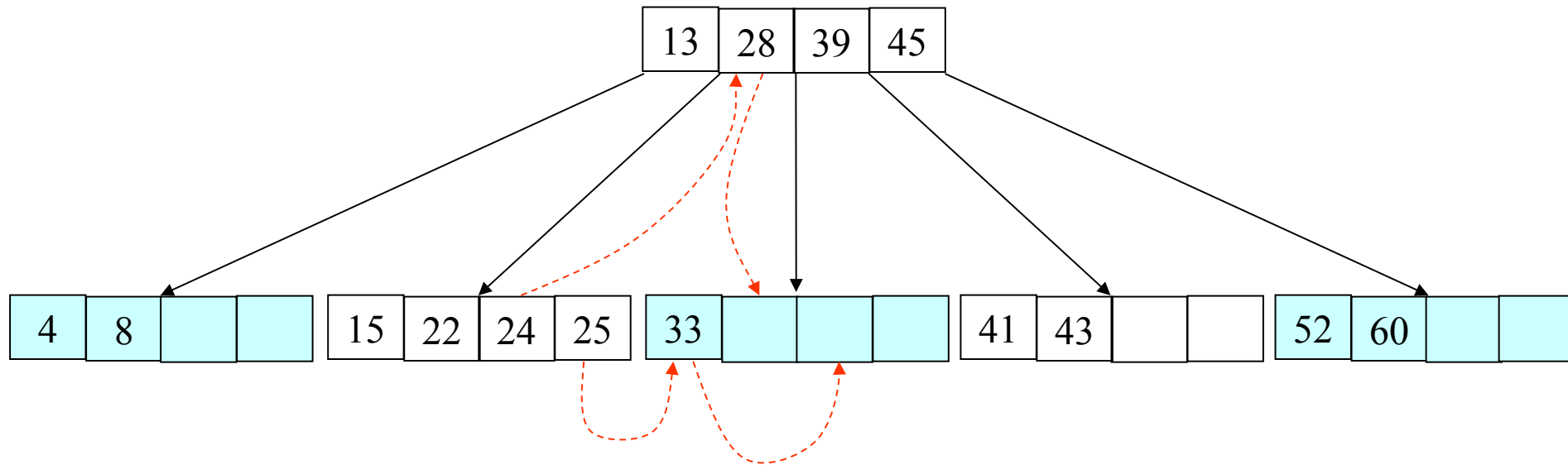
El elemento está en una
pagina hoja. No es
necesario sustituirlo

Pero no cumple la
condición de un árbol B

6. Eliminación

Se tiene un árbol B con los siguientes elementos:

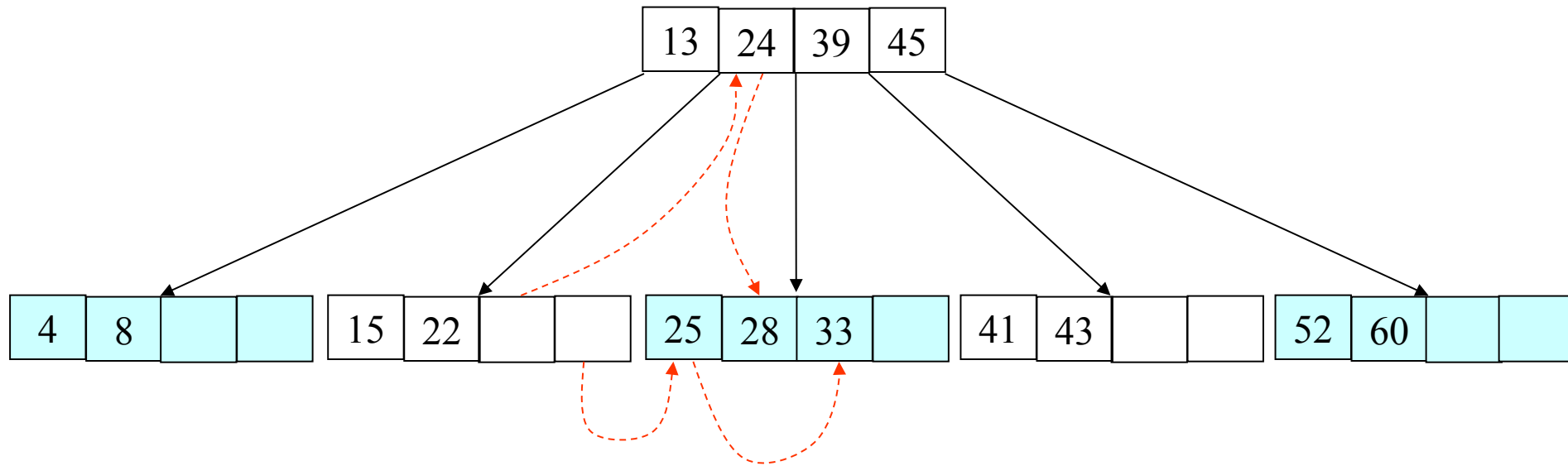
Eliminar: 35



6. Eliminación

Se tiene un árbol B con los siguientes elementos:

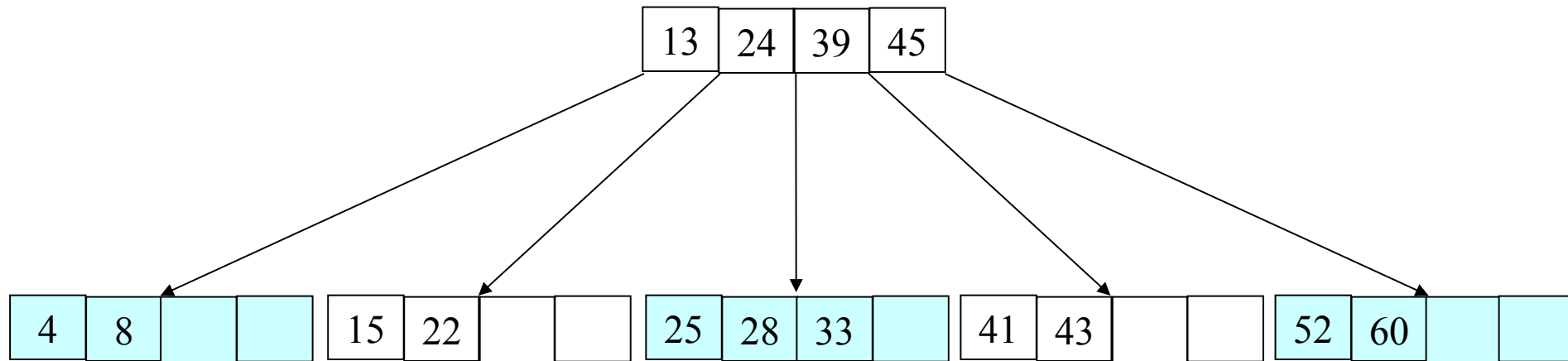
Eliminar: 35



6. Eliminación

Se tiene un árbol B con los siguientes elementos:

Se ha eliminado: 35

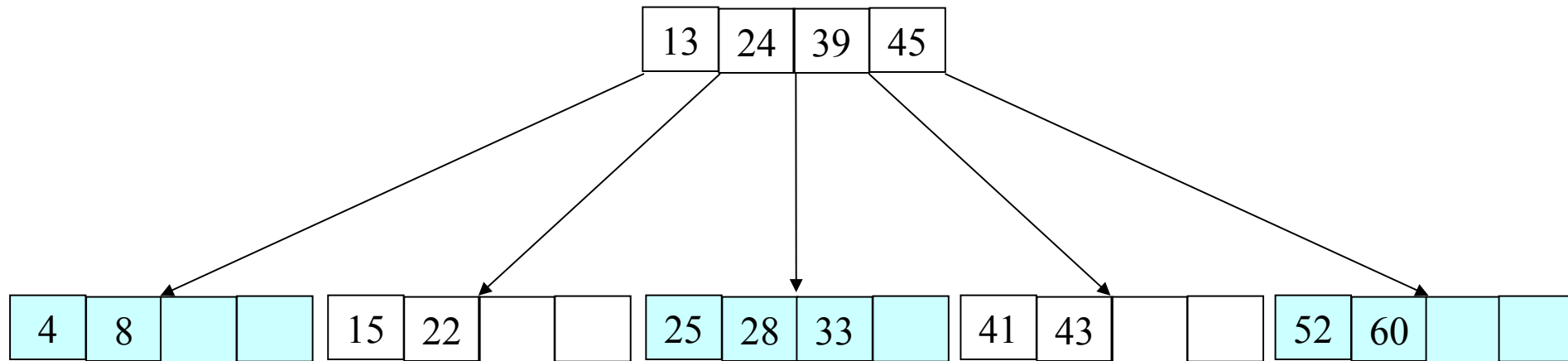


Cumple condiciones de
un árbol B

6. Eliminación

Se tiene un árbol B con los siguientes elementos:

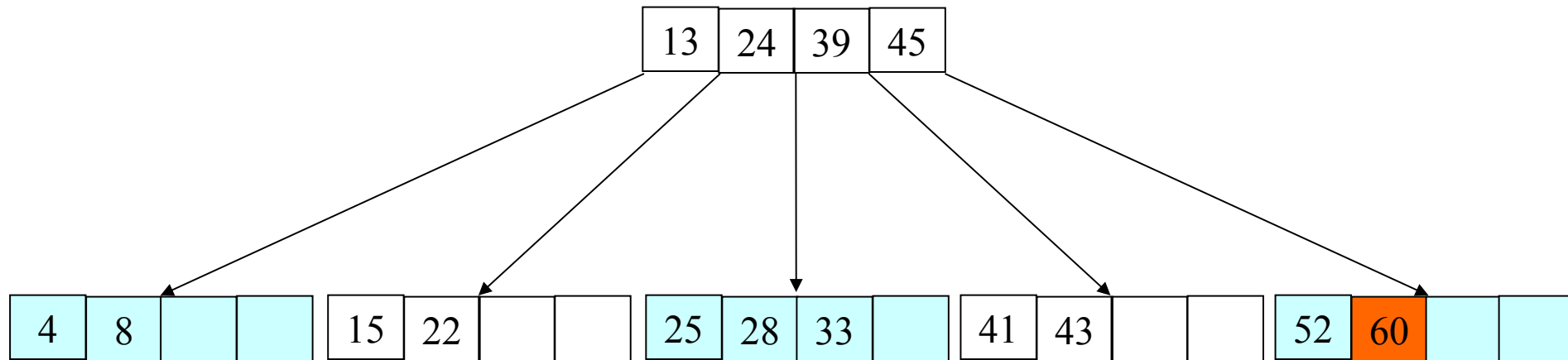
Eliminar: 60



6. Eliminación

Se tiene un árbol B con los siguientes elementos:

Eliminar: 60

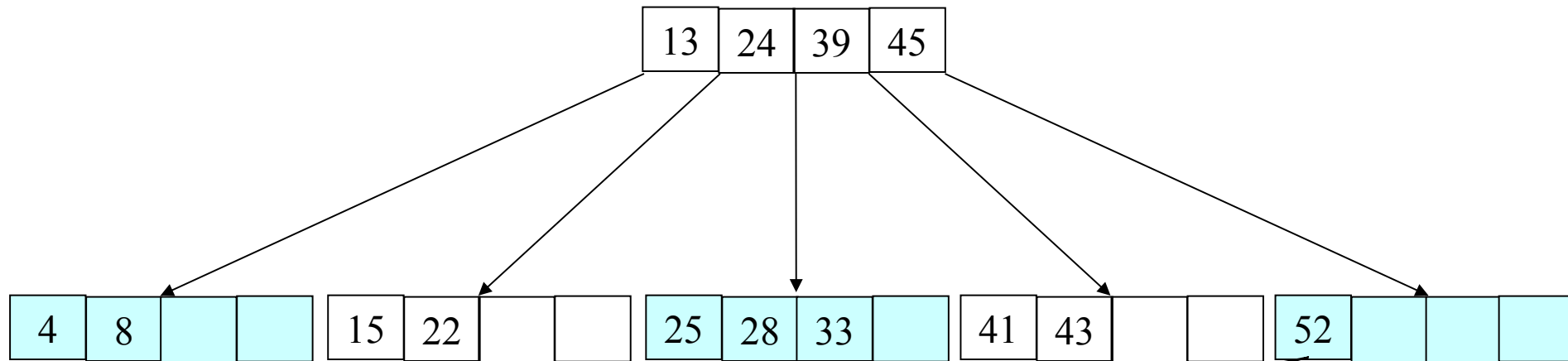


El elemento esta en una
pagina hoja. No es
necesario sustituirlo

6. Eliminación

Se tiene un árbol B con los siguientes elementos:

Eliminar: 60

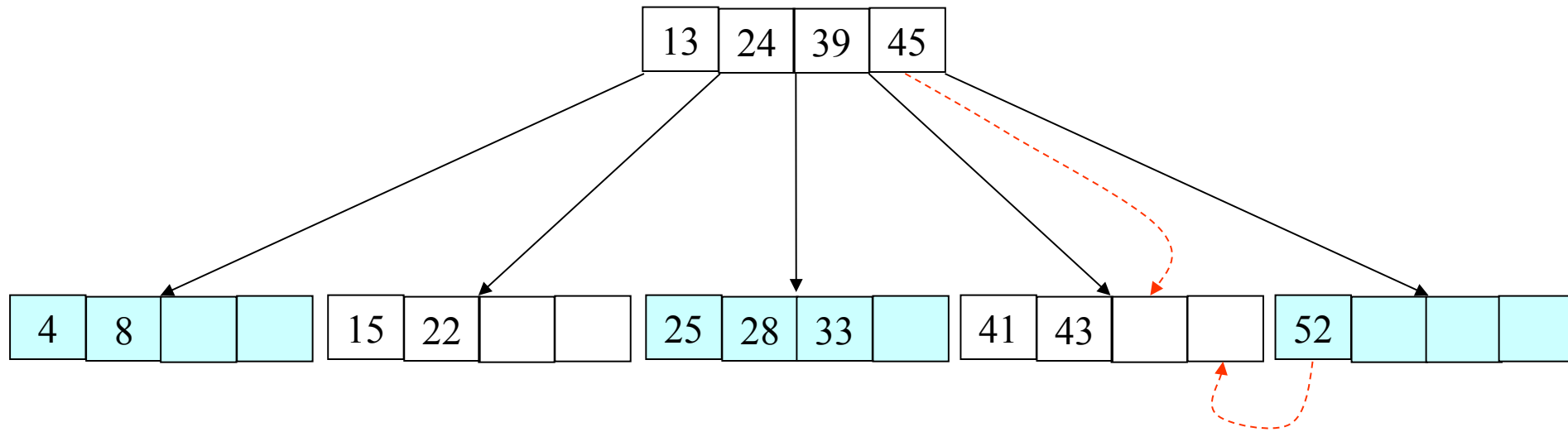


Pero no cumple la
condición de un árbol B

6. Eliminación

Se tiene un árbol B con los siguientes elementos:

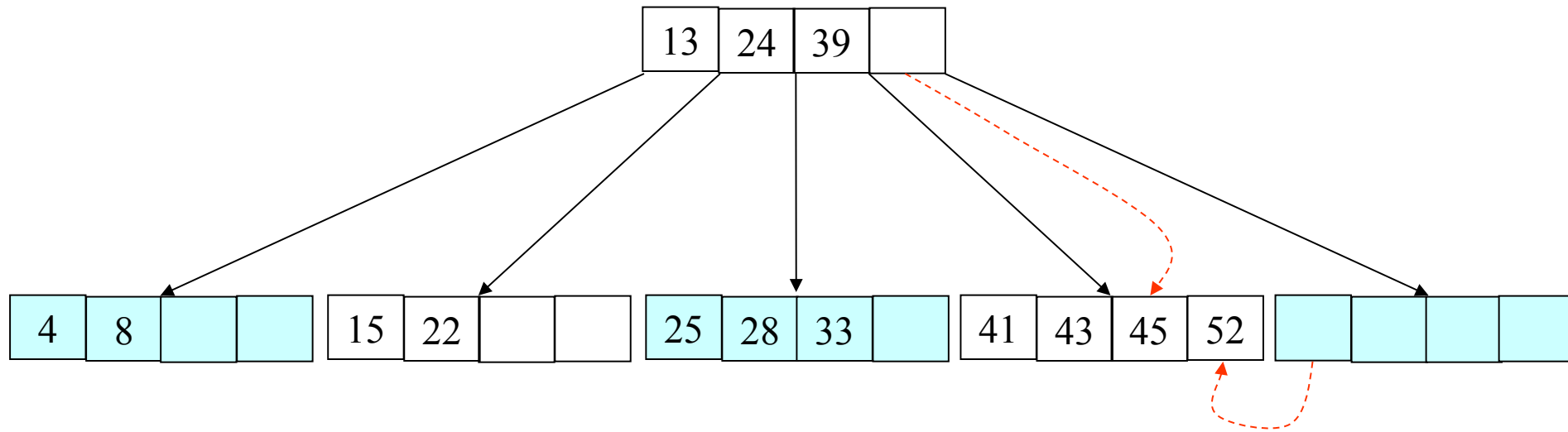
Eliminar: 60



6. Eliminación

Se tiene un árbol B con los siguientes elementos:

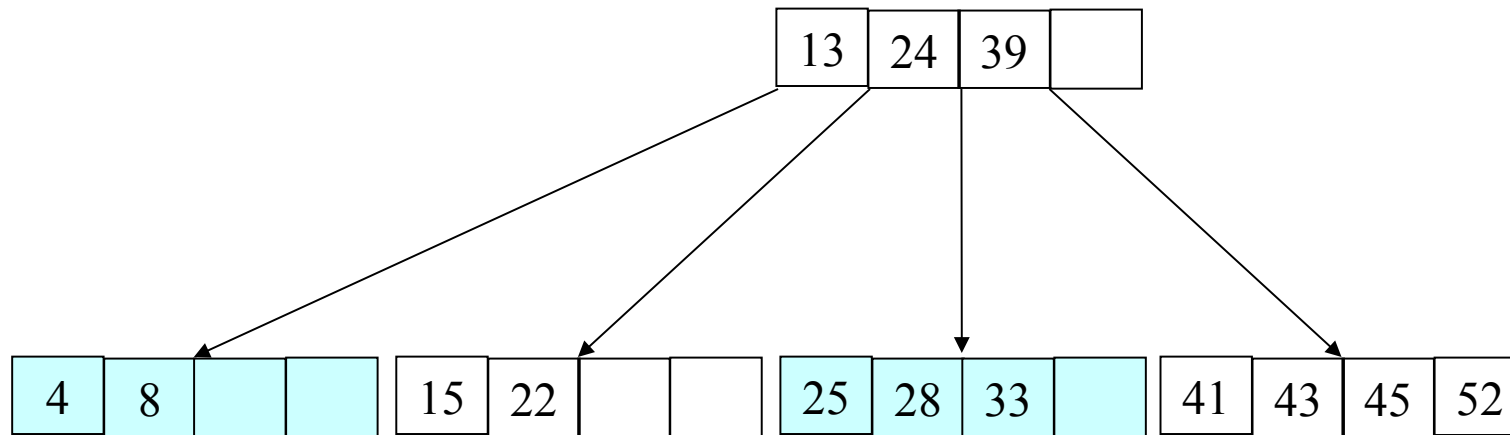
Eliminar: 60



6. Eliminación

Se tiene un árbol B con los siguientes elementos:

Se Eliminó: 60



Cumple condiciones de
un árbol B

7. Árboles B^+

La principal característica de estos árboles es que todas las claves se encuentran en las hojas. Ocupan mas espacio que los árboles B, pero es aceptable si se modifica continuamente, pues se evita la reorganización del árbol que es muy costosa en los árboles B

Referencias Bibliográficas

- Cairo Battistutti, Osvaldo y Guardati Buemo, Silvia. (1994) Estructura de Datos. McGraw-Hill. Mexico D.F.
- Joyanes Aguilar, Luis y Zahonero Martínez, Ignacio. (2004) Algoritmos y Estructuras de Datos en C. McGraw-Hill. Madrid.
- Goodrich T. Michael y Tamasia, Roberto (2002). Estructura de Datos y Algoritmos en Java. Grupo Patria Cultural. Mexico D. F.